Foundations of Spatial AI

Lecture 11 — Relightable Representations

Lecture notes (Giacomini, Di Giammarino – Sapienza University of Rome)

October 28, 2026

Contents

1 Relightable 3D Representations *(slides 1–6)* **2**

1.1 Recap: Neural 3D Representations *(slide 3)* . . . 2

1.2 Radiance Fields Bake In the Lighting *(slide 4)* . . . 2

1.3 Goal: Relightable 3D *(slide 5)* . . . 3

1.4 Today's Goals *(slide 6)* . . . 4

2 What We Need: The Rendering Equation *(slides 7–9)* **4**

2.1 Material: How a Surface Reflects Light *(slide 8)* . . . 4

2.2 Light: What Arrives at Each Point *(slide 9)* . . . 5

3 Let's Learn Everything We Need *(slides 10–11)* **5**

3.1 Learn Geometry, Material and Light with Networks *(slide 10)* . . . 5

3.2 Rendering a Pixel *(slide 11)* . . . 6

4 Does It Work? Why Not? *(slides 12–17)* **6**

4.1 Does It Work? *(slide 12)* . . . 6

4.2 Why? What One Capture Observes *(slide 13)* . . . 7

4.3 Many Explanations, Same Pixels *(slides 14–16)* . . . 8

4.4 It Happens on Real Objects *(slide 17)* . . . 8

5 How Do We Resolve the Ambiguity? *(slide 18)* **8**

6 Option 1: Constrain the Model *(slides 19–34)* **10**

6.1 Constrain the Light *(slide 20)* . . . 10

6.2 Constrain the Material *(slide 21)* . . . 10

6.3 Constraints Help Even with Sparse Views *(slides 22–23)* . . . 11

6.4 PhySG: The Full Pipeline *(slide 24)* . . . 11

6.5 Spherical Gaussians Make the Integral Closed-Form *(slide 25)* . . . 12

6.6 PhySG: Results *(slide 26)* . . . 13

6.7 What PhySG Ignores: Shadows & Interreflections *(slide 27)* . . . 13
6.8 Recall: Gaussians Store Baked Color *(slide 28)* . . . 13
6.9 Relightable Gaussians (IRGS): Store Material *(slide 29)* . . . 14
6.10 Blend Attributes, Then Shade *(slide 30)* . . . 14
6.11 What About Shadows? *(slide 31)* . . . 15
6.12 Putting Shadows in the Rendering Equation *(slide 32)* . . . 16
6.13 IRGS: Results *(slides 33–34)* . . . 16

7 Option 2: Capture More Informative Data *(slides 35–41)* **16**
7.1 Capture: One Light at a Time (OLAT) *(slide 36)* . . . 16
7.2 Light Adds Up *(slides 37–38)* . . . 17
7.3 Light Transport Is Linear *(slide 39)* . . . 18
7.4 Relighting = a Weighted Sum *(slides 40–41)* . . . 18

8 Option 3: Learn Priors from Data *(slides 42–47)* **19**
8.1 MaterialFusion: Prior Inside the Optimization *(slide 43)* . . . 19
8.2 Neural Gaffer: Learn to Relight Images Directly *(slide 45)* . . . 20
8.3 How Do You Tell a Network the Light? *(slide 46)* . . . 20
8.4 Neural Gaffer: Data, 3D and Limits *(slide 47)* . . . 21

9 Summary *(slide 48)* **21**

1 Relightable 3D Representations *(slides 1–6)*

1.1 Recap: Neural 3D Representations *(slide 3)*

Every neural 3D representation seen so far (density fields such as NeRF and 3D Gaussian Splatting, and neural surfaces such as SDFs used in IDR) can be summarised as a function that, given a 3D point and a viewing direction, returns a colour.

Definition 1.1 (Radiance-field representation)**.** A *radiance-field representation* is a parametrised function

$$g_\phi(\boldsymbol{x}, \boldsymbol{\omega}) = \boldsymbol{c} \in \mathbb{R}^3,$$

mapping a point $\boldsymbol{x} \in \mathbb{R}^3$ and a direction $\boldsymbol{\omega} \in \mathbb{S}^2$ to the colour $\boldsymbol{c}$ observed when looking at $\boldsymbol{x}$ from direction $\boldsymbol{\omega}$. Density fields (NeRF, 3DGS) and neural surfaces (SDFs, e.g. IDR) differ in *where* they place matter, but both output $\boldsymbol{c}$ in this way.

1.2 Radiance Fields Bake In the Lighting *(slide 4)*

In these models the colour is the light *emitted* (leaving the point) towards the camera. The question is whether the model can be rendered under a different illumination. The answer is no: the lighting under which the photos were taken is baked into g_ϕ.

[Insert image from slide 4]

Figure 1: Radiance fields bake in the lighting (NeRF ship, IDR cans). Take the four images of slide 4. Figure credits: Mildenhall et al., ECCV 2020 (NeRF); Yariv et al., NeurIPS 2020 (IDR).

Proposition 1.2 (No relighting from a radiance field)**.** *The function g_ϕ has no input or parameter that represents the illumination. Hence the only image-space family it can produce is the one under the capture light; changing the light requires changing ϕ, and nothing in ϕ says* how*.*

Intuition

Think of a photograph of a statue under a desk lamp: the bright cheek and the dark side of the nose are *painted* into the photo. A radiance field stores this painted result at every point. To move the lamp you would need to know the statue's real colour (the *albedo*), its shape and how shiny it is — exactly the ingredients the field never separated.

1.3 Goal: Relightable 3D *(slide 5)*

The goal is to start from posed images under *one unknown light* and produce a model that can render novel views *and* novel illuminations (relit images under "light 1", "light 2", . . .).

[Insert image from slide 5]

Figure 2: Goal of relightable 3D: novel views, relit under light 1, relit under light 2. Take the three images of slide 5. Figure credits: Zhang et al., CVPR 2021 (PhySG).

[Insert image from slide 2]

Figure 3: Three renderings of the same cow under different illuminations (title slide of the topic). Take the three cows of slide 2. Figure credits: S. Tulsiani.

1.4 Today's Goals *(slide 6)*

1. What a relightable model needs.
2. Why learning it from images is ambiguous.
3. Three ways to resolve the ambiguity: *constrain the model*, *capture more informative data*, *learn priors from data*.

2 What We Need: The Rendering Equation *(slides 7–9)*

Definition 2.1 (Rendering equation, reflection form)**.** Let $\boldsymbol{x}$ be a point on a surface with unit normal $\boldsymbol{n}$, Ω the hemisphere of directions around $\boldsymbol{n}$, and $\boldsymbol{\omega}_o$ the outgoing direction. The radiance leaving $\boldsymbol{x}$ towards $\boldsymbol{\omega}_o$ is

$$L_o(\boldsymbol{x}, \boldsymbol{\omega}_o) = \int_\Omega \underbrace{f_r(\boldsymbol{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o)}_{\text{material}} \underbrace{L_i(\boldsymbol{x}, \boldsymbol{\omega}_i)}_{\text{light}} \underbrace{(\boldsymbol{n}^\mathsf{T}\boldsymbol{\omega}_i)}_{\text{geometry}} \, \mathrm{d}\boldsymbol{\omega}_i.$$

The three coloured factors correspond to the three unknowns of the problem: *material* (f_r), *light* (L_i) and *geometry* (the normal $\boldsymbol{n}$ in the cosine foreshortening term, together with the surface position $\boldsymbol{x}$).

Proposition 2.2 (Relighting by swapping the light)**.** *If material f_r and geometry $(\boldsymbol{x}, \boldsymbol{n})$ are known, then the image under any new light L_i^{new} is*

$$L_o^{\text{new}}(\boldsymbol{x}, \boldsymbol{\omega}_o) = \int_\Omega f_r(\boldsymbol{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o)\, L_i^{\text{new}}(\boldsymbol{x}, \boldsymbol{\omega}_i)\, (\boldsymbol{n}^\mathsf{T}\boldsymbol{\omega}_i)\, \mathrm{d}\boldsymbol{\omega}_i.$$

This is exact for direct *light only; indirect light (light bounced from other surfaces) depends on the scene and must be recomputed.*

Intuition

The equation is a recipe with three ingredients. If you know the ingredients "material" and "geometry", then "light" is just a plug-in slot: pull out the old light, plug in a new one, re-evaluate the integral. The caveat is that part of the light arriving at $\boldsymbol{x}$ comes from other objects in the scene, whose own appearance changes with the new light.

2.1 Material: How a Surface Reflects Light *(slide 8)*

Definition 2.3 (BRDF)**.** The *bidirectional reflectance distribution function* $f_r(\boldsymbol{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) \geq 0$ tells how much of the light arriving from direction $\boldsymbol{\omega}_i$ leaves towards $\boldsymbol{\omega}_o$. It is a function on $\mathbb{S}^2 \times \mathbb{S}^2$ (four degrees of freedom) at every surface point.

A *shiny* material concentrates the reflected light in a narrow lobe around the mirror direction; a *rough* material spreads it widely (Figure 4).

Example 2.4 (Lambertian material)**.** A *Lambertian* (perfectly diffuse) surface has $f_r(\boldsymbol{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) = a(\boldsymbol{x})/\pi$, where $a(\boldsymbol{x}) \in [0,1]^3$ is the *albedo*. Then

$$L_o(\boldsymbol{x}, \boldsymbol{\omega}_o) = \frac{a(\boldsymbol{x})}{\pi} \int_\Omega L_i(\boldsymbol{x}, \boldsymbol{\omega}_i)\, (\boldsymbol{n}^\mathsf{T}\boldsymbol{\omega}_i)\, \mathrm{d}\boldsymbol{\omega}_i = \frac{a(\boldsymbol{x})}{\pi} E(\boldsymbol{x}),$$

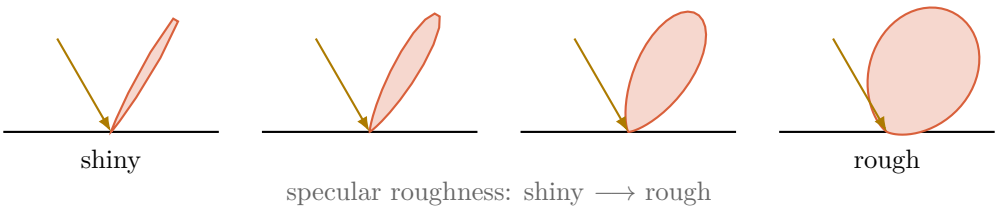


Figure 4: Schematic BRDF lobes for a fixed incoming direction (yellow arrow): the reflected energy (orange) is concentrated around the mirror direction for shiny materials and spreads out as roughness grows.

independent of $\boldsymbol{\omega}_o$, where $E(\boldsymbol{x})$ is the irradiance. Under constant light $L_i \equiv 1$ one has $E = \int_\Omega \cos\theta \, \mathrm{d}\boldsymbol{\omega}_i = \pi$, hence $L_o = a(\boldsymbol{x})$: the albedo is the image of a surface under uniform unit illumination.

2.2 Light: What Arrives at Each Point *(slide 9)*

The incident radiance $L_i(\boldsymbol{x}, \boldsymbol{\omega}_i)$ is light arriving at $\boldsymbol{x}$ from *every* direction of the hemisphere, and it differs at every point because of occlusion, nearby objects, etc.

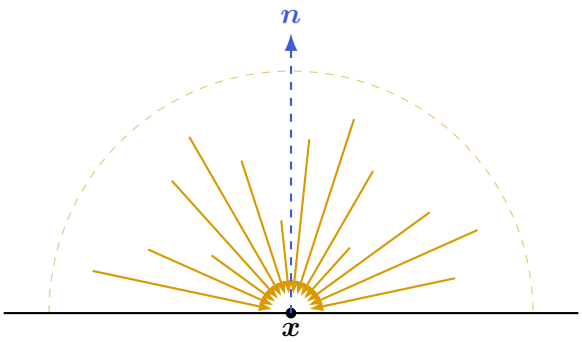


Figure 5: Incoming radiance $L_i(\boldsymbol{x}, \boldsymbol{\omega}_i)$ at a surface point $\boldsymbol{x}$: one value per direction of the hemisphere (arrow length $\propto$ radiance). The normal $\boldsymbol{n}$ is dashed.

Intuition

Standing in a room, the light reaching your eye is not a single number: the window is bright, the wall opposite is dim, the ceiling in-between. $L_i(\boldsymbol{x}, \cdot)$ is a "panorama of brightness" seen from $\boldsymbol{x}$, and the surface multiplies it by the BRDF and sums it up. Standing under a table gives a different panorama than standing next to it.

3 Let's Learn Everything We Need *(slides 10–11)*

3.1 Learn Geometry, Material and Light with Networks *(slide 10)*

Same setup as NeRF: posed images under one unknown illumination. Each unknown is a network: an SDF network for geometry, a BRDF network for material, a light network for the light. A renderer integrates the rendering equation, and all three networks are trained jointly with a photometric loss.

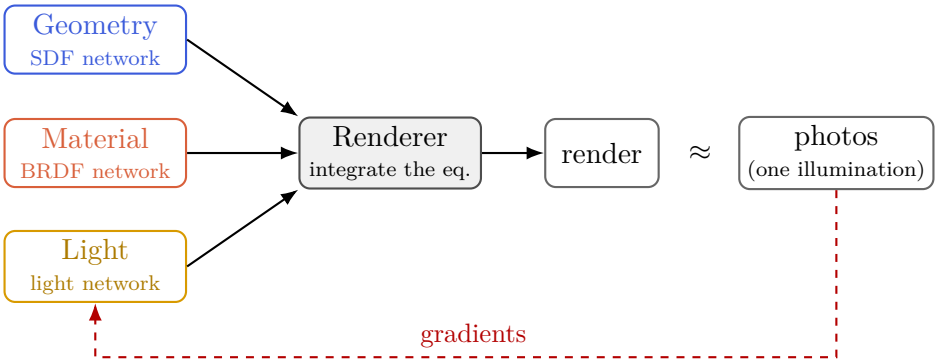


Figure 6: Inverse rendering by analysis-by-synthesis: three networks feed a differentiable renderer; the photometric error is back-propagated to all of them.

3.2 Rendering a Pixel *(slide 11)*

The integral in Definition 2.1 is evaluated by Monte Carlo sampling.

Proposition 3.1 (Monte Carlo estimator)**.** *Let* $\boldsymbol{\omega}_i^{(1)}, \dots, \boldsymbol{\omega}_i^{(N)} \sim p$ *be i.i.d. directions on* Ω *with density* $p > 0$*. Then*

$$\widehat{L}_o(\boldsymbol{x}, \boldsymbol{\omega}_o) = \frac{1}{N} \sum_{j=1}^{N} \frac{f_r(\boldsymbol{x}, \boldsymbol{\omega}_i^{(j)}, \boldsymbol{\omega}_o)\, L_i(\boldsymbol{x}, \boldsymbol{\omega}_i^{(j)})\, (\boldsymbol{n}^\mathsf{T} \boldsymbol{\omega}_i^{(j)})}{p(\boldsymbol{\omega}_i^{(j)})}$$

is an unbiased estimator of $L_o(\boldsymbol{x}, \boldsymbol{\omega}_o)$*.*

Proof. By linearity of expectation, $\mathbb{E}[\widehat{L}_o] = \mathbb{E}_{\boldsymbol{\omega}_i \sim p}\left[\frac{h(\boldsymbol{\omega}_i)}{p(\boldsymbol{\omega}_i)}\right] = \int_\Omega \frac{h}{p}\, p \, \mathrm{d}\boldsymbol{\omega}_i = \int_\Omega h \, \mathrm{d}\boldsymbol{\omega}_i = L_o$, with $h = f_r L_i (\boldsymbol{n}^\mathsf{T} \boldsymbol{\omega}_i)$. □

Algorithm 3.2 (Rendering a pixel)**.**
1. Find the surface point $\boldsymbol{x}$ along the pixel ray (sphere tracing on the SDF).
2. Sample N incoming directions $\boldsymbol{\omega}_i^{(j)} \sim p$.
3. Query the BRDF network and the light network.
4. Average the weighted samples (Proposition 3.1), compare to the photo, back-propagate.

Intuition

Instead of summing over *all* directions (impossible), poll N random ones, ask "how much light comes from there and how much of it is sent to the camera?", and average. Dividing by p corrects for having sampled some directions more often than others, so the answer is right on average.

4 Does It Work? Why Not? *(slides 12–17)*

4.1 Does It Work? *(slide 12)*

On a synthetic cow, the fit (free per-pixel albedo plus a free environment map, fitted to one photo) reproduces the training view with error below 1/255. But under a *moving light* the fit is wrong: shading has been painted into the albedo.

[Insert image from slide 12]

Figure 7: Photo (one illumination), fit, and moving light: truth vs. fit. Take the four cow images of slide 12. Figure credits: S. Tulsiani.

Example 4.1 (What "error $< 1/255$" means)**.** For pixel values in $[0, 1]$, a maximum error of $1/255$ gives MSE $\leq (1/255)^2$ and PSNR $= 10 \log_{10}(1/\text{MSE}) \geq 20 \log_{10} 255 \approx 48.1$ dB: the training view is reproduced essentially perfectly, while the relit result is still wrong. Training error therefore says nothing about relighting quality.

Proposition 4.2 (Trivial explanation: shading painted on, flat light)**.** *Consider a Lambertian scene and an observed image $I(\boldsymbol{x})$ that is independent of the viewing direction. Take albedo $a'(\boldsymbol{x}) = I(\boldsymbol{x})$ and a uniform environment $L_e \equiv 1$. Then the model reproduces every training image exactly.*

Proof. By the Lambertian example, under $L_e \equiv 1$ the rendered radiance equals the albedo, $L_o = a'(\boldsymbol{x}) = I(\boldsymbol{x})$. □

4.2 Why? What One Capture Observes *(slide 13)*

Each surface point is seen under **one** light, from a *few* directions. The unknowns are a full BRDF **and** all the incoming light.

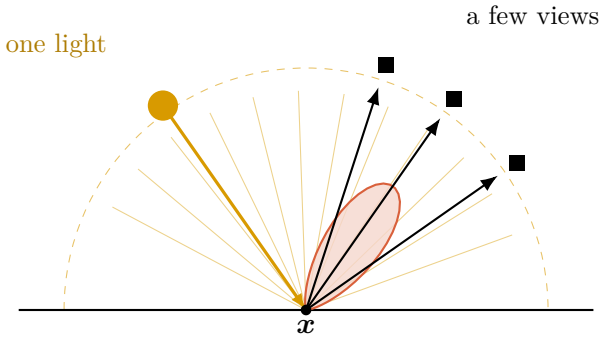


Figure 8: What a single capture observes at $\boldsymbol{x}$: one light direction (thick yellow), a handful of view directions (black squares). The thin yellow rays and the orange lobe remain unconstrained.

Proposition 4.3 (Counting unknowns)**.** *At a surface point with K views under one light, the data are K colour values, whereas the unknowns include a BRDF (a function of 4 angular variables) and an incident-light function (2 angular variables). The unknowns outnumber the data by an unbounded factor, so the inverse problem is under-determined.*

4.3 Many Explanations, Same Pixels *(slides 14–16)*

Proposition 4.4 (Factorisation ambiguity)**.** *For a Lambertian pixel $I = a \odot s$ (albedo a, shading/light term s, both in $\mathbb{R}^3_{>0}$) and any $c \in \mathbb{R}^3_{>0}$, the pair $(a \oslash c,\ c \odot s)$ produces the same pixel.*

Proof. $(a \oslash c) \odot (c \odot s) = a \odot s = I$. □

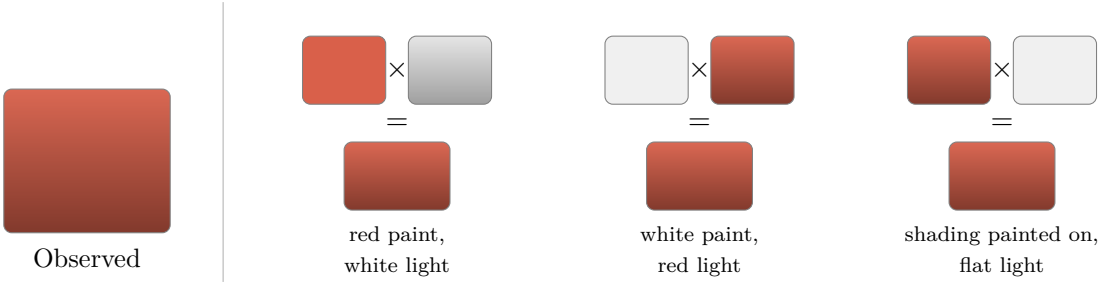


Figure 9: Same pixels, many (material, light) explanations. Left: the observed shaded red object. Right: three different factorisations (albedo × light) producing it.

Example 4.5 (Three explanations, one pixel)**.** Let the observed RGB pixel be (0.80, 0.35, 0.30). The following all give it, channel by channel:

explanation	albedo a	light term s
red paint, white light	$(0.80, 0.35, 0.30)$	$(1, 1, 1)$
white paint, red light	$(1, 1, 1)$	$(0.80, 0.35, 0.30)$
shading painted on, flat light	per-pixel $= I$	$(1, 1, 1)$ (flat)

Only the first one relights correctly if the light is turned white→blue.

Intuition

It is the same puzzle as "is the car red, or is it white under a red light?". A photo only records the *product* albedo × light. Like trying to recover the two factors of 12 from the number 12 alone: 3×4, 2×6 and 12×1 are all fine, and nothing in the product tells you which one is true.

4.4 It Happens on Real Objects *(slide 17)*

On Stanford-ORB with the *same* method (nvdiffrecmc) and similar recovered shapes, the relighting PSNR of two objects is 24.4 dB versus 7.5 dB.

Example 4.6 (17 dB is a lot)**.** Since $\text{PSNR} = -10 \log_{10} \text{MSE}$, a gap of $24.4 - 7.5 = 16.9$ dB means a mean-squared-error ratio of $10^{1.69} \approx 49$: the relit failure is about fifty times worse in squared error, with no sign of it in the training views.

5 How Do We Resolve the Ambiguity? *(slide 18)*

Three levers, each acting on a different part of the inverse-rendering loop:

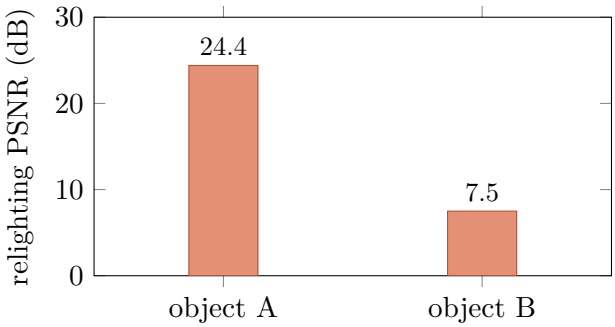


Figure 10: Same method, similar shapes, very different relighting quality (values from the slide; the two objects are those of the Stanford-ORB figure).

[Insert image from slide 17]

Figure 11: Baking and Salt objects: ground truth and prediction, novel view and relighting. Take the full 8-image grid of slide 17. Figure credits: Kuang et al., NeurIPS 2023 (Stanford-ORB).

1. **Constrain the model** (on the unknowns).
2. **Capture more (informative) data** (on the photos).
3. **Learn priors from data** (a learned prior over the unknowns).

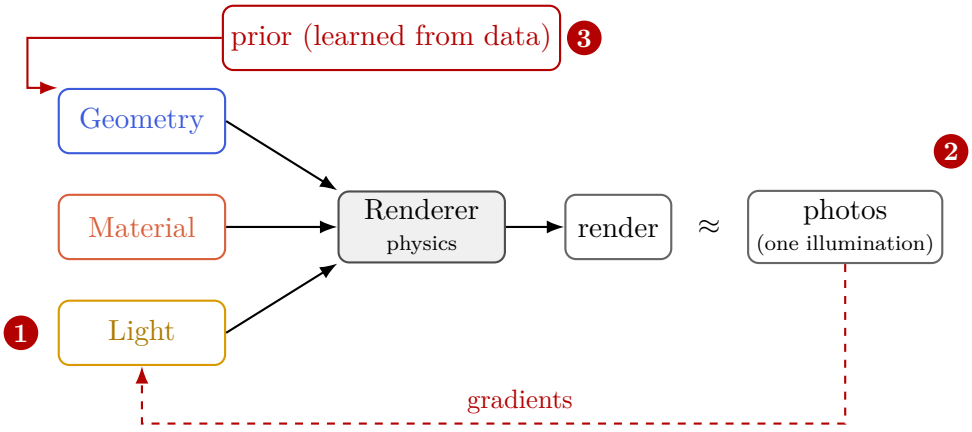


Figure 12: The three strategies: **1** constrain what the unknowns can be, **2** capture photos that carry more information, **3** inject a prior learned from data.

6 Option 1: Constrain the Model *(slides 19–34)*

6.1 Constrain the Light *(slide 20)*

Definition 6.1 (Distant (environment) light)**.** Light is *far away* if it depends only on direction, not on position:

$$L_i(\boldsymbol{x}, \boldsymbol{\omega}) \approx L_e(\boldsymbol{\omega}) \qquad \forall \boldsymbol{x},$$

where $L_e : \mathbb{S}^2 \to \mathbb{R}^3_{\geq 0}$ is an *environment map*, one per scene. Often it is additionally taken to have *no colour* (a gray environment map, as in NeRS).

A gray environment map removes the colour ambiguity that the NeRS paper states as "to avoid the fundamental ambiguity (a red car vs. a white car under red light)": it rules out explanation 2 of the previous section (white paint, red light).

Proposition 6.2. *Under a colourless light $L_e = \ell(\boldsymbol{\omega})\,(1,1,1)^\mathsf{T}$, the factorisation of Proposition 4.4 is restricted to scalar $c \in \mathbb{R}_{>0}$, so the chromatic part of the pixel colour is unambiguously assigned to albedo.*

Proof. Colourless light means $s = \sigma\,(1,1,1)^\mathsf{T}$ for a scalar σ; the transformed pair $c \odot s$ stays colourless only if c has equal entries. The ratios between colour channels of $a \odot s$ are then those of a. □

[Insert image from slide 20]

Figure 13: Twelve panoramic environment maps (one per scene). Take the 4×3 grid of slide 20. Figure credits: Zhang et al., NeurIPS 2021 (NeRS); Poly Haven (env maps).

6.2 Constrain the Material *(slide 21)*

Definition 6.3 (Diffuse + specular BRDF)**.**

$$f_r(\boldsymbol{x}, \boldsymbol{\omega}_i, \boldsymbol{\omega}_o) = \underbrace{\frac{a(\boldsymbol{x})}{\pi}}_{\substack{\text{diffuse: varies over the surface}}} + \underbrace{f_s(\boldsymbol{\omega}_i, \boldsymbol{\omega}_o; \rho)}_{\substack{\text{specular: few params, often shared}}},$$

with ρ the specular roughness (shiny → rough, see Figure 4).

The albedo (texture) still varies from point to point, but the specular part is often a single material for the whole object. In parameter count, a free BRDF per point has four angular variables of freedom; the constrained model has one albedo (3 numbers) per point plus one shared roughness.

Intuition

Most real objects are "a coloured paint plus one kind of gloss". By committing to this, we replace an infinite-dimensional unknown at every point by a handful of numbers, which the few available views can actually pin down.

6.3 Constraints Help Even with Sparse Views *(slides 22–23)*

A constrained BRDF generalises; free-form appearance overfits. In NeRS (cars from a few images) the constrained model gives a clean result, while a version with a free radiance field, IDR and NeRF degrade.

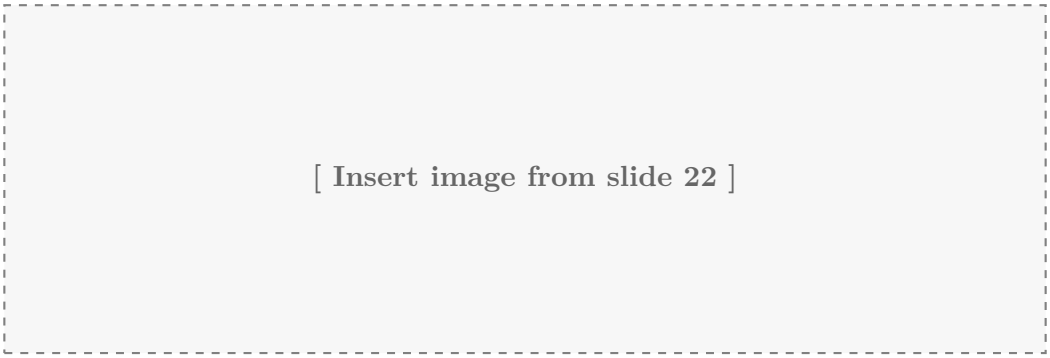


Figure 14: Sparse-view reconstruction: input image, NeRS, NeRS with radiance field, IDR, NeRF (two cars). Take the image grid of slide 22. Video credits: Zhang et al., NeurIPS 2021 (NeRS).

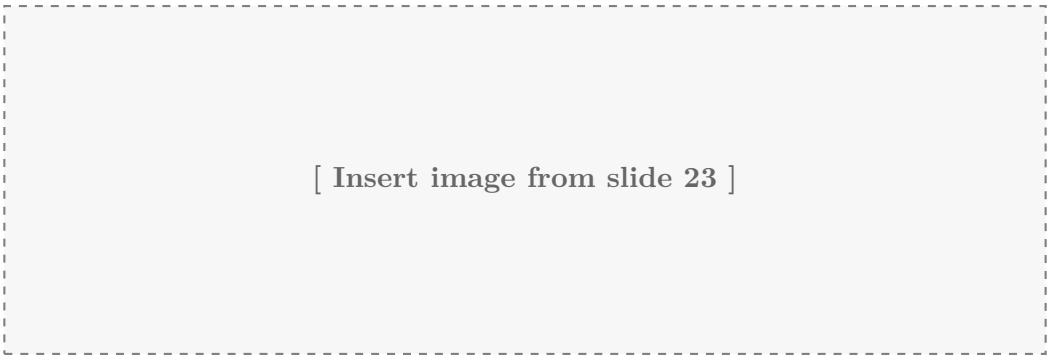


Figure 15: NeRS: photo, gray-albedo estimated light, and estimated environment map (three cars). Take the 3×3 grid of slide 23. Figure credits: Zhang et al., NeurIPS 2021 (NeRS), Fig. 10.

6.4 PhySG: The Full Pipeline *(slide 24)*

PhySG instantiates all of the above in one model:

Geometry	SDF network, sphere tracing (as in lecture 09)
Material	albedo network + **one** shared specular lobe
Light	env map = 128 spherical Gaussians
Integral	closed form (below); no shadows

6.5 Spherical Gaussians Make the Integral Closed-Form *(slide 25)*

Definition 6.4 (Spherical Gaussian)**.** A *spherical Gaussian* (SG) with axis $\boldsymbol{\xi} \in \mathbb{S}^2$, sharpness $\lambda > 0$ and amplitude $\boldsymbol{\mu} \in \mathbb{R}^3_{\geq 0}$ is

$$G(\boldsymbol{\omega}) = \boldsymbol{\mu}\, e^{\lambda(\boldsymbol{\xi}^\mathsf{T}\boldsymbol{\omega}-1)}.$$

An environment map is approximated by a sum of SG lobes.

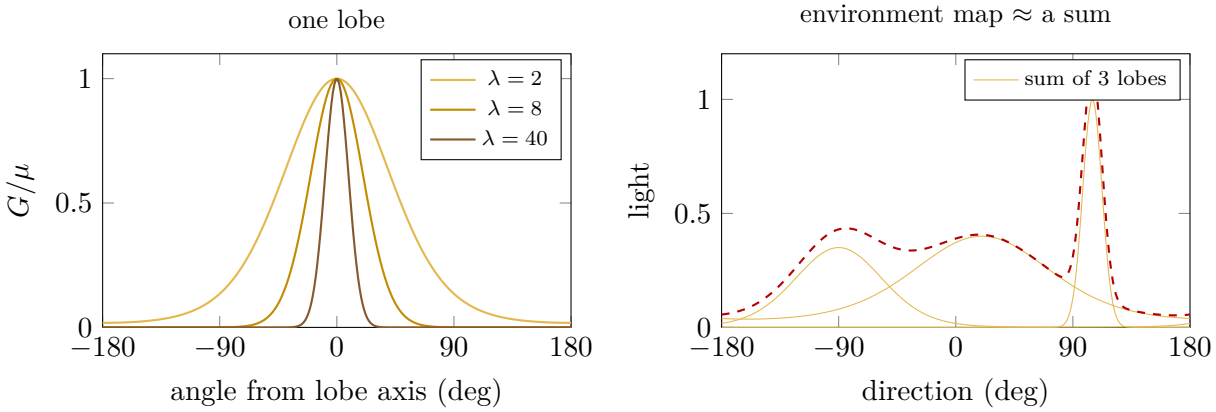


Figure 16: Left: SG lobes for increasing sharpness λ (narrower lobe). Right: three SG lobes (thin) add up to a smooth-plus-peaked environment (dashed).

Lemma 6.5 (Product of two SGs is an SG)**.** *For* $G_k(\boldsymbol{\omega}) = \boldsymbol{\mu}_k e^{\lambda_k(\boldsymbol{\xi}_k^\mathsf{T}\boldsymbol{\omega}-1)}$, $k = 1, 2$, *one has* $G_1 G_2 = G_3$ *with*

$$\lambda_3 = \|\lambda_1\boldsymbol{\xi}_1 + \lambda_2\boldsymbol{\xi}_2\|, \quad \boldsymbol{\xi}_3 = \frac{\lambda_1\boldsymbol{\xi}_1 + \lambda_2\boldsymbol{\xi}_2}{\lambda_3}, \quad \boldsymbol{\mu}_3 = \boldsymbol{\mu}_1 \odot \boldsymbol{\mu}_2\, e^{\lambda_3-\lambda_1-\lambda_2}.$$

Proof. The exponent of the product is $\lambda_1(\boldsymbol{\xi}_1^\mathsf{T}\boldsymbol{\omega} - 1) + \lambda_2(\boldsymbol{\xi}_2^\mathsf{T}\boldsymbol{\omega} - 1) = (\lambda_1\boldsymbol{\xi}_1 + \lambda_2\boldsymbol{\xi}_2)^\mathsf{T}\boldsymbol{\omega} - \lambda_1 - \lambda_2 = \lambda_3\boldsymbol{\xi}_3^\mathsf{T}\boldsymbol{\omega} - \lambda_1 - \lambda_2 = \lambda_3(\boldsymbol{\xi}_3^\mathsf{T}\boldsymbol{\omega} - 1) + (\lambda_3 - \lambda_1 - \lambda_2)$. □

Lemma 6.6 (Integral of an SG)**.** $\displaystyle\int_{\mathbb{S}^2} G(\boldsymbol{\omega})\,\mathrm{d}\boldsymbol{\omega} = \boldsymbol{\mu}\,\frac{2\pi}{\lambda}(1 - e^{-2\lambda})$.

Proof. In spherical coordinates about $\boldsymbol{\xi}$ with $t = \cos\theta$: $\int_0^{2\pi}\int_{-1}^{1} \boldsymbol{\mu}\, e^{\lambda(t-1)}\,\mathrm{d}t\,\mathrm{d}\phi = 2\pi\boldsymbol{\mu}\, e^{-\lambda}\frac{e^{\lambda}-e^{-\lambda}}{\lambda}$. □

Theorem 6.7 (Closed-form shading with SGs)**.** *Let the light be* $L_e = \sum_{k=1}^{K} G_k$*, let the specular lobe* f_s *be an SG in* $\boldsymbol{\omega}$ *(at fixed* $\boldsymbol{\omega}_o$ *and* $\boldsymbol{n}$*) and let the clamped cosine be approximated by an SG* G_c*. Then*

$$\int_\Omega f_r\, L_e\, (\boldsymbol{n}^\mathsf{T}\boldsymbol{\omega}_i)\,\mathrm{d}\boldsymbol{\omega}_i \;\approx\; \sum_{k=1}^{K}\int_{\mathbb{S}^2} G_k\, G_s\, G_c\,\mathrm{d}\boldsymbol{\omega} = \sum_{k=1}^{K} \boldsymbol{\mu}'_k \frac{2\pi}{\lambda'_k}(1 - e^{-2\lambda'_k}),$$

where $(\boldsymbol{\mu}'_k, \lambda'_k)$ *are obtained by applying Lemma 6.5 twice. The cost per surface point is* $O(K)$*, with no sampling.*

Proof. Linearity in L_e turns the integral into a sum over lobes; each term is an integral of a product of three SGs, which is a single SG by Lemma 6.5 (applied twice), integrated by Lemma 6.6. □

Intuition

Gaussians are closed under multiplication — the reason every Gaussian-based tool (Kalman filters, Gaussian processes) is so convenient. Here, "light × shiny lobe × cosine" multiplies into just one more lobe, whose total energy we know by formula. So instead of sampling hundreds of directions, PhySG evaluates $K = 128$ short formulas.
Worked numbers: a lobe with $\lambda = 2$ integrates to $\pi(1 - e^{-4}) \approx 3.08\,\mu$, $\lambda = 8$ to $\approx 0.79\,\mu$, $\lambda = 40$ to $\approx 0.157\,\mu$. A lobe covers roughly $2\pi/\lambda$ steradians of the 4π sphere — the sharper, the smaller.

6.6 PhySG: Results *(slide 26)*

[Insert image from slide 26]

Figure 17: PhySG: novel views, diffuse-only edit, relit under light 1 and light 2 (fish figurine). Take the four images of slide 26. Video credits: Zhang et al., CVPR 2021 (PhySG).

6.7 What PhySG Ignores: Shadows & Interreflections *(slide 27)*

A shadow means less light *arriving* at $\boldsymbol{x}$. PhySG's integral has no visibility term, so the only way to explain the darkness of a shadow is a darker albedo. The same holds for interreflections. The fix is shadow rays (next).

Proposition 6.8 (Shadows are absorbed by albedo)**.** *Let the true model be $L_o = \frac{a}{\pi} V\, E$ with visibility $V \in [0, 1]$ and unshadowed irradiance E. A model that omits V fits the same pixel with albedo $a' = aV$.*

Proof. $\frac{a'}{\pi} E = \frac{aV}{\pi} E = L_o$. □

Example 6.9. True albedo $a = 0.8$ and a point in the shadow of an occluder blocking 75% of the light: $V = 0.25$. The model without visibility reproduces the photo with $a' = 0.2$. Under a new light that comes from the other side, this point would be rendered much too dark.

6.8 Recall: Gaussians Store Baked Color *(slide 28)*

In 3D Gaussian Splatting a pixel is $\sum_i w_i\, \boldsymbol{c}_i(\boldsymbol{\omega})$. Each Gaussian stores *outgoing* light, baked in like NeRF: same problem, colour is light leaving the point.

[Insert image from slide 27]

Figure 18: Direct vs. indirect illumination, and albedo/roughness with and without modelling indirect illumination (hotdog scene). Take the full figure of slide 27. Figure credits: Zhang et al., CVPR 2022 (InvRender); Hasselgren et al., NeurIPS 2022.

6.9 Relightable Gaussians (IRGS): Store Material *(slide 29)*

IRGS uses 2D Gaussians (flat disks), so each disk has a well-defined normal. Each Gaussian carries a **normal**, an **albedo** and a **roughness**; the light is one environment map, as before.

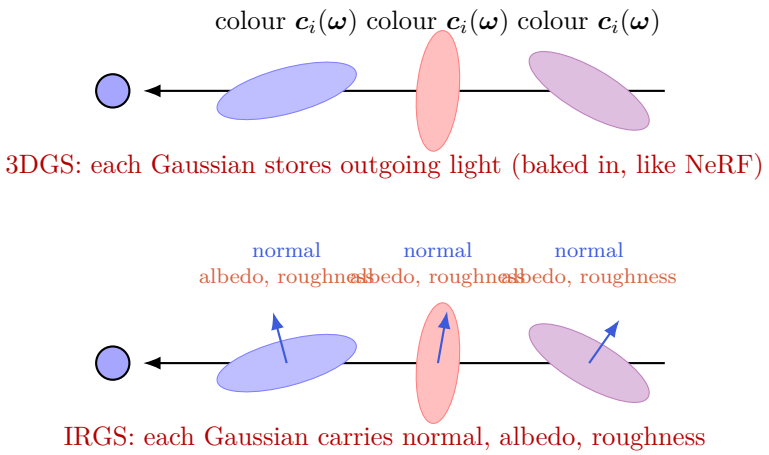


Figure 19: Top: 3DGS Gaussians store a baked colour. Bottom: IRGS Gaussians are disks with a well-defined normal and store material properties instead.

6.10 Blend Attributes, Then Shade *(slide 30)*

Definition 6.10 (Per-pixel attribute blending)**.** With front-to-back splatting weights w_i for the Gaussians covering a pixel, the pixel receives

$$\bar{\boldsymbol{n}} = \sum_i w_i\, \boldsymbol{n}_i, \qquad \bar{a} = \sum_i w_i\, a_i, \qquad \bar{r} = \sum_i w_i\, r_i, \qquad \bar{d} = \sum_i w_i\, d_i,$$

the blended normal (renormalised), albedo, roughness and depth d_i. The surface point is $\boldsymbol{x} = \boldsymbol{o} + \bar{d}\,\boldsymbol{\omega}$ for camera centre $\boldsymbol{o}$ and pixel ray direction $\boldsymbol{\omega}$, and the pixel colour is $\mathrm{shade}(\boldsymbol{x}, \bar{\boldsymbol{n}}, \bar{a}, \bar{r}; \mathrm{light})$.

Intuition

Shading is expensive (hundreds of rays), so we do it *once per pixel*, not once per Gaussian. First blend the "G-buffer" (normal, albedo, roughness, depth) exactly like a deferred

renderer, then light the resulting surface point.

[Insert image from slide 30]

Figure 20: Blended per-pixel attributes of the cow: normal, albedo, roughness, depth $\to \boldsymbol{x}$, and the shaded result. Take the five cow images of slide 30. Figure credits: Gu et al., CVPR 2025 (IRGS).

6.11 What About Shadows? *(slide 31)*

From $\boldsymbol{x}$ (at the blended depth), trace through the Gaussians: the accumulated opacity gives the visibility $V(\boldsymbol{x}, \boldsymbol{\omega})$; and IRGS also gets **indirect light**: the colour of whatever the ray hits.

Lemma 6.11 (Visibility from opacity)**.** *Along a ray crossing Gaussians with opacities* $\alpha_1, \ldots, \alpha_n$ *(front to back), set* $T_i = \prod_{j<i}(1-\alpha_j)$ *and* $w_i = \alpha_i T_i$*. Then* $\sum_{i=1}^{n} w_i = 1 - \prod_{i=1}^{n}(1-\alpha_i)$*, so the visibility* V *(the fraction of light that is* not *absorbed) satisfies* $V = 1 - \sum_i w_i = \prod_i (1-\alpha_i)$*.*

Proof. Induction on n. For $n=1$: $w_1 = \alpha_1 = 1-(1-\alpha_1)$. If it holds for n, then $\sum_{i\le n+1} w_i = 1 - T_{n+1} + \alpha_{n+1}T_{n+1} = 1 - T_{n+1}(1-\alpha_{n+1}) = 1 - T_{n+2}$. □

Example 6.12. Two Gaussians with $\alpha = (0.5, 0.4)$: $w = (0.5,\ 0.4 \cdot 0.5 = 0.2)$, $\sum w = 0.7$, and $V = 0.5 \cdot 0.6 = 0.3 = 1 - 0.7$. Only 30% of the light gets through.

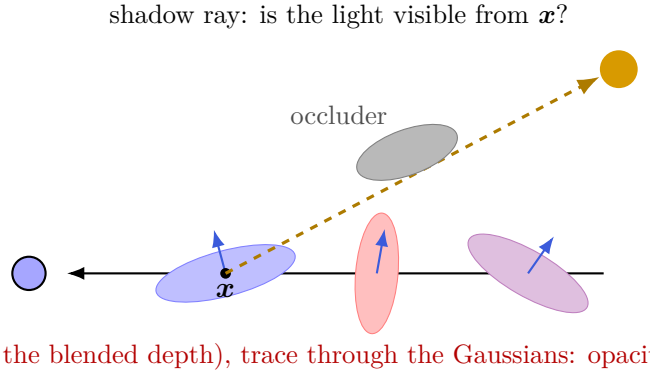


Figure 21: Shadow ray from the blended surface point $\boldsymbol{x}$. If it crosses other Gaussians, their accumulated opacity reduces the visibility V; the colour of what it hits gives indirect light.

6.12 Putting Shadows in the Rendering Equation *(slide 32)*

Proposition 6.13 (IRGS shading estimator)**.** *With N sampled directions $\boldsymbol{\omega}_j \sim p$,*

$$L_o \approx \frac{1}{N} \sum_j f_r(\boldsymbol{\omega}_j, \boldsymbol{\omega}_o) \Big[\underbrace{V(\boldsymbol{x}, \boldsymbol{\omega}_j)}_{\text{traced opacity}} \; L_e(\boldsymbol{\omega}_j) + \underbrace{L_{\text{ind}}(\boldsymbol{x}, \boldsymbol{\omega}_j)}_{\text{traced colour}} \Big] \frac{\boldsymbol{n}^\mathsf{T} \boldsymbol{\omega}_j}{p(\boldsymbol{\omega}_j)}.$$

Opacity gives visibility V; colour gives indirect light (the Gaussians' own colour). The cost is 256 rays per pixel, on a random subset of pixels only.

Intuition

The environment light L_e is now only counted when the ray escapes (V close to 1); when the ray hits something, we use that something's colour instead — a one-bounce approximation of global illumination, implemented by the very same Gaussians. Tracing rays on just a random subset of pixels per step keeps training affordable.

Algorithm 6.14 (IRGS pixel rendering)**.**

1. Splat Gaussians: obtain blended $\bar{\boldsymbol{n}}, \bar{a}, \bar{r}, \bar{d}$ and weights w_i.
2. Compute $\boldsymbol{x} = \boldsymbol{o} + \bar{d}\boldsymbol{\omega}$.
3. Sample directions $\boldsymbol{\omega}_j$ (256 per pixel on a random pixel subset), trace each through the Gaussians to get $V(\boldsymbol{x}, \boldsymbol{\omega}_j)$ and $L_{\text{ind}}(\boldsymbol{x}, \boldsymbol{\omega}_j)$.
4. Evaluate the BRDF from $(\bar{a}, \bar{r})$ and sum as above; compare with the photo and back-propagate.

6.13 IRGS: Results *(slides 33–34)*

[Insert image from slide 33]

Figure 22: Global, direct and indirect illumination: the two terms of the equation rendered separately (toy scene). Take the three images of slide 33. Video credits: Gu et al., CVPR 2025 (IRGS).

7 Option 2: Capture More Informative Data *(slides 35–41)*

7.1 Capture: One Light at a Time (OLAT) *(slide 36)*

A *light stage* moves a light over the sphere of directions and takes one photo per light direction (about 2048 directions in roughly a minute). Instead of one illumination, we observe each point under thousands.

[Insert image from slide 34]

Figure 23: IRGS materials and relighting: albedo, roughness, normal and five environment maps for four scenes. Take the 4×8 grid of slide 34. Figure credits: Gu et al., CVPR 2025 (IRGS).

[Insert image from slide 36]

Figure 24: Light stage capture: grid of OLAT photos and the lighting environment. Take the video frame of slide 36. Video credits: Debevec et al., SIGGRAPH 2000 (Light Stage).

7.2 Light Adds Up *(slides 37–38)*

Theorem 7.1 (Superposition of light)**.** *For a fixed camera and scene, and linear (HDR) pixel values, let I_A and I_B be the images under lights A and B and I_{A+B} the image with both on. Then $I_{A+B} = I_A + I_B$, and more generally $I(c_A A + c_B B) = c_A I_A + c_B I_B$.*

Proof. The full light-transport operator (including shadows, interreflections and any number of bounces) is linear in the emitted radiance of the sources, because photons do not interact with each other: each is propagated independently and the contributions add. □

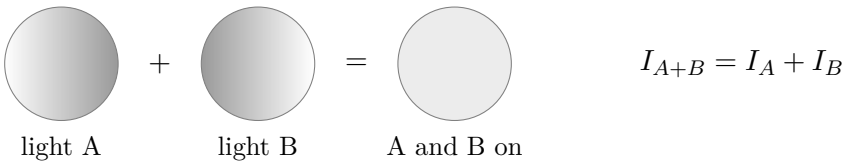


Figure 25: Light adds up: the image under two lights at once is the sum of the two single-light images (schematic sphere lit from the left, from the right, and from both).

[Insert image from slide 38]

Figure 26: Real photos of a face under two lights and their sum. Take the video frame of slide 38. Video credits: Debevec et al., SIGGRAPH 2000 (Light Stage).

7.3 Light Transport Is Linear *(slide 39)*

Definition 7.2 (Light-transport matrix)**.** Discretise the light into K single-direction lights with intensities ℓ_k and let $I_k \in \mathbb{R}^P$ be the (vectorised) image under light k alone. The *light-transport matrix* is $\boldsymbol{T} = [I_1 \,|\, \cdots \,|\, I_K] \in \mathbb{R}^{P \times K}$ and the image under lighting $\boldsymbol{\ell}$ is

$$I = \sum_k \ell_k I_k = \boldsymbol{T}\boldsymbol{\ell}.$$

$\boldsymbol{T}$ already holds material, geometry and shadows. The assumption: fixed camera and scene, linear HDR pixels.

7.4 Relighting = a Weighted Sum *(slides 40–41)*

Algorithm 7.3 (Relighting from OLAT images)**.**

1. Capture $\{I_k\}_{k=1}^K$, one per light direction $\boldsymbol{\omega}_k$.
2. For a new environment map L_e^{new}, set $\ell_k = L_e^{\text{new}}(\boldsymbol{\omega}_k)\,\Delta\boldsymbol{\omega}_k$ ($\Delta\boldsymbol{\omega}_k$ the solid angle of direction k).
3. Output $I_{\text{new}} = \sum_k \ell_k I_k$.

No BRDF anywhere, and no geometry: the result is just a sum.

Example 7.4. Take a pixel whose OLAT values at three lights are (0.80, 0.30, 0.50) and a new light with weights (0.06, 0.90, 0.03). Then $I_{\text{new}} = 0.048 + 0.270 + 0.015 = 0.333$. With $K = 2048$ images of one megapixel in RGB, the table $\boldsymbol{T}$ holds about $2048 \cdot 3 \cdot 10^6 \approx 6.1 \cdot 10^9$ numbers.

Intuition

Imagine a dark room with 2048 tiny lamps, each lit alone for one photo. Any new lighting is a mixing board: "a bit of lamp 17, a lot of lamp 340, . . . " and the final photo is the same mix of the stored photos. You never needed to understand the object; the light stage did the physics for you. The price is the rig itself, and a static subject.

[Insert image from slide 41]

Figure 27: Relighting with the light stage: weights from a captured environment map and the relit faces. Take the video frame and the strip of five faces on slide 41. Video credits: Debevec et al., SIGGRAPH 2000 (Light Stage).

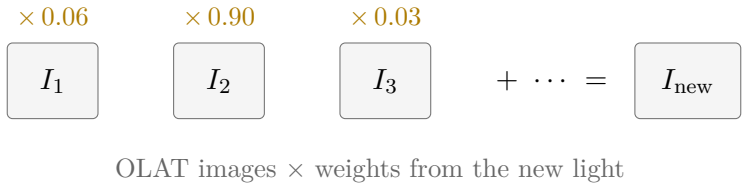


Figure 28: Relighting as a weighted sum of OLAT images (weights are those of slide 40, six out of 64 lights shown there).

8 Option 3: Learn Priors from Data *(slides 42–47)*

8.1 MaterialFusion: Prior Inside the Optimization *(slide 43)*

Keep the physical model and per-scene optimisation of Option 1, but add a learned prior on the *materials*: a material diffusion model scores how plausible the rendered albedo and roughness maps are.

Definition 8.1 (MaterialFusion objective)**.** Given posed photos under one illumination, fit mesh, materials and environment map θ by

$$\min_{\theta} \; \mathcal{L}_{\text{photo}}(\theta) + \gamma \, \mathcal{L}_{\text{prior}}(\text{albedo}_\theta, \; \text{roughness}_\theta),$$

where $\mathcal{L}_{\text{photo}}$ compares renders and photos, and $\mathcal{L}_{\text{prior}}$ is a loss derived from the material diffusion model (low when the rendered albedo and roughness look like plausible materials); $\gamma > 0$ sets the strength of the prior.

Algorithm 8.2 (MaterialFusion)**.**
1. Input: posed photos, one illumination.
2. Fit mesh, materials and environment map to the photos (photometric loss).
3. Render albedo and roughness; the material diffusion model scores their plausibility → prior loss.
4. Back-propagate the sum into θ.

Intuition

The photo loss alone accepts any explanation that matches the pixels (the previous section's puzzle); the prior is a friend who has seen thousands of real materials and says "a can of baking powder doesn't have a glossy sheet of random shading painted on its albedo". In probabilistic language the prior plays the role of $-\log p(\text{materials})$ added to the data term.

Priors and per-scene optimisation are not exclusive: you get both.

[Insert image from slide 43]

Figure 29: MaterialFusion pipeline on a cow: posed photos, fitted rendering, rendered albedo and roughness. Take the images of slide 43. Figure credits: Litman et al., 3DV 2025 (MaterialFusion); S. Tulsiani.

[Insert image from slide 44]

Figure 30: Relighting a real can: reference, MaterialFusion, nvdiffrecmc (same optimisation without the prior). Take the video frame of slide 44. Video credits: Litman et al., 3DV 2025 (MaterialFusion).

8.2 Neural Gaffer: Learn to Relight Images Directly *(slide 45)*

Neural Gaffer skips the explicit BRDF and light estimate entirely: one image plus a target environment map goes in, a relit image comes out. A diffusion model learns the mapping.

[Insert image from slide 45]

Figure 31: Neural Gaffer: input images and relit results under environment maps and text-conditioned lights. Take the full grid of slide 45. Video credits: Jin et al., NeurIPS 2024 (Neural Gaffer).

8.3 How Do You Tell a Network the Light? *(slide 46)*

The environment map is converted into two images (an LDR tone-mapped copy and a log-normalised copy), both *rotated into the camera frame*, VAE-encoded and concatenated with the encoded input image; the denoising U-Net (Zero123 weights, fine-tuned) produces the relit image.

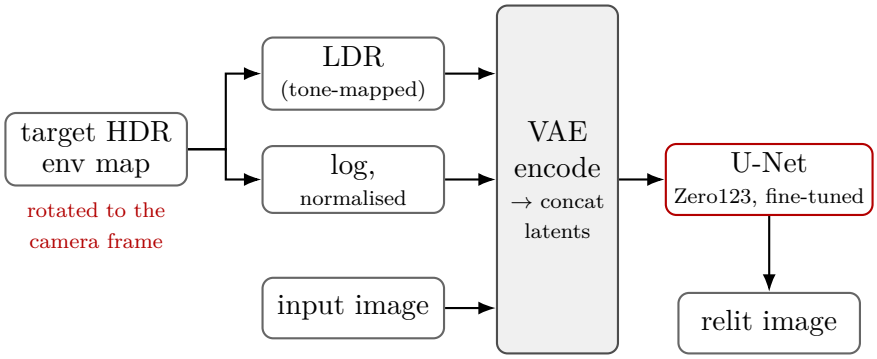


Figure 32: Neural Gaffer conditioning: each env-map pixel is made to mean a fixed direction in the camera frame.

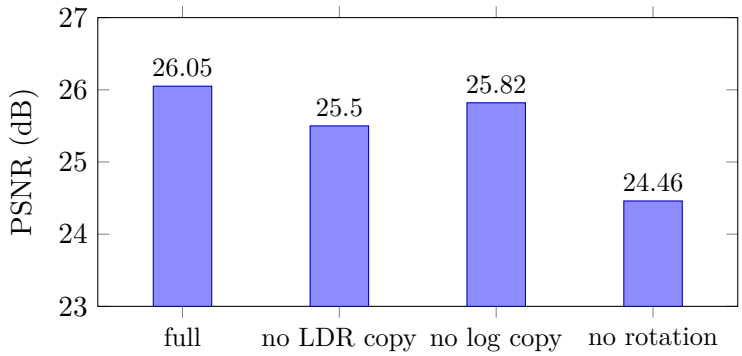


Figure 33: Neural Gaffer ablation (PSNR, values from the slide; the vertical axis starts at 23 dB). Removing the rotation into the camera frame hurts most (−1.59 dB).

Intuition

For a network, "the light" must be a fixed code. If the same env-map pixel meant "north" in one example and "to the camera's left" in another, the network could not learn a consistent rule; expressing the map in the camera frame gives every pixel a stable meaning, which is why dropping the rotation costs the most. The two copies help for a different reason: the LDR copy keeps the visible look, the log copy keeps the dynamic range of very bright lamps.

8.4 Neural Gaffer: Data, 3D and Limits *(slide 47)*

- **Data:** about 90K objects and 18.4M renders, i.e. roughly 18.4M/90K ≈ 200 renders per object (many views × many lights).
- **3D:** relight the views, then refit the NeRF (once per target light).
- **Limits:** only 256^2 images; and a yellow object or a white one under yellow light? For diffuse objects the ambiguity remains.

9 Summary *(slide 48)*

One illumination → many (material, light) explanations. The three remedies: **1** constrain the model (distant light, few-parameter BRDF, traced visibility); **2** capture more (light is linear → relight by summing OLAT images); **3** priors (learn plausible materials, or learn to relight directly).

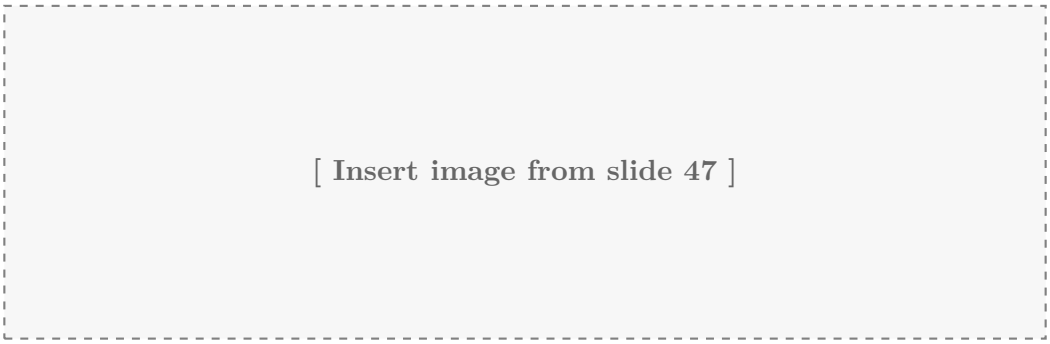


Figure 34: 3D relighting comparison: input views, nvdiffrecmc, TensoIR, Neural Gaffer (ours), ground truth and target lighting (three objects). Take the full grid of slide 47. Figure credits: Jin et al., NeurIPS 2024 (Neural Gaffer).

Method	**Option**	**Light / material model**	**Shadows, indirect**	**Needs**	**Main limitation**
NeRF, 3DGS, IDR	–	$g_\phi(\boldsymbol{x}, \boldsymbol{\omega}) = \boldsymbol{c}$, colour = emitted light	–	posed images	cannot be relit: light baked in
NeRS	1	gray env map; constrained BRDF	not covered	few images of an object	light and BRDF kept simple
PhySG	1	128-SG env map; albedo net + one shared specular lobe; SDF geometry	none (no visibility term)	posed images, one light	shadows end up in the albedo
IRGS	1	env map; per-Gaussian normal, albedo, roughness	shadow rays through Gaussians; indirect colour	posed images; 256 rays/pixel on random pixel subset	ray-tracing cost
Light stage (OLAT)	2	none: $I = \boldsymbol{T}\boldsymbol{\ell}$	already in $\boldsymbol{T}$	light-stage rig, fixed camera and scene, linear HDR	special capture setup
MaterialFusion	3	physical model + diffusion prior on albedo and roughness	via the physical renderer	posed photos, one light; per-scene optimisation	per-scene optimisation
Neural Gaffer	3	none explicit: diffusion model (image + env map $\rightarrow$ relit image)	implicit	one image + target env map; trained on $\sim$90K objects, 18.4M renders	only 256^2; diffuse colour/light ambiguity

Table 1: Comparison of the methods covered in this lecture.

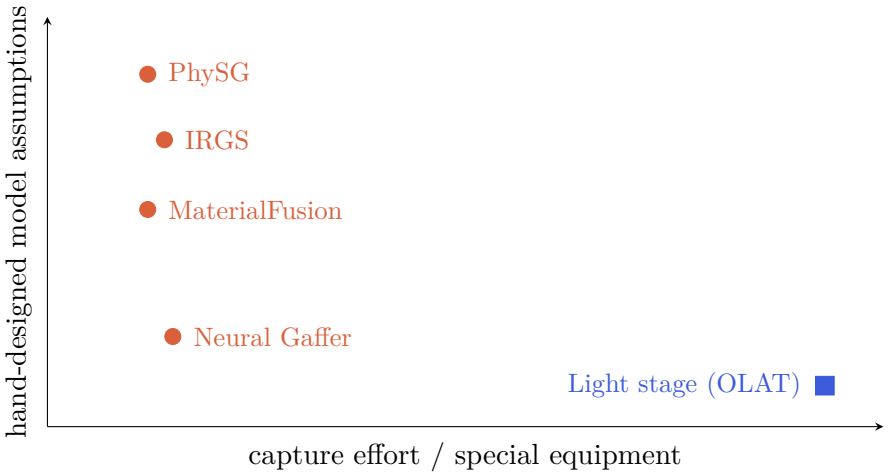


Figure 35: Schematic (qualitative, not measured) positioning of the methods: the light stage shifts the burden to the capture hardware and needs almost no model assumptions; Option 1 methods need only ordinary photos but rely on strong hand-designed assumptions; priors (Option 3) trade assumptions for learned knowledge, at the price of training data.

Key take-aways

1. Radiance fields store *emitted* colour, so the capture lighting is baked in. Relighting needs the separate ingredients of the rendering equation: geometry, material (BRDF) and light.
2. Fitting them from photos under one illumination is under-determined: a pixel only records the *product* of albedo and light, so many (material, light) explanations fit the training views perfectly yet fail under a new light.
3. **Constrain the model:** a distant, often colourless light, a few-parameter BRDF with a shared specular lobe, and (PhySG) spherical Gaussians for a closed-form integral; adding traced visibility and indirect light (IRGS) stops shadows from leaking into the albedo.
4. **Capture more:** since light transport is linear, OLAT images relight by a weighted sum with no BRDF and no geometry, but it requires a special capture setup.
5. **Learn priors:** either inside the optimisation (MaterialFusion's plausibility loss on albedo and roughness) or by learning to relight directly (Neural Gaffer), which is still limited by resolution and by the residual diffuse colour/light ambiguity.

Acknowledgments. The lecture is adapted from *Learning for 3D Vision*, CMU 16-825 (Prof. Shubham Tulsiani). Cow illustrations by S. Tulsiani; light-stage material from Debevec et al., SIGGRAPH 2000; results and diagrams from Mildenhall et al. (NeRF), Yariv et al. (IDR), Zhang et al. (PhySG, NeRS, InvRender), Kuang et al. (Stanford-ORB), Wang et al. (SIGGRAPH Asia 2009), Hasselgren et al. (nvdiffrecmc), Gu et al. (IRGS), Litman et al. (MaterialFusion), Jin et al. (Neural Gaffer); environment maps from Poly Haven.